

- расширить WorkstationUIState
- добавить supportedActions в WorkstationUIState
- добавить selectedActionIndex в WorkstationUIState
- добавить selectedRecipeIndex в WorkstationUIState
- добавить statusMessage в WorkstationUIState
- подтягивать uiTitle из WorkstationDatabase
- подтягивать supportedActions из WorkstationDatabase
- убрать главный switch как основной источник UI-данных
- оставить fallback на случай отсутствия WorkstationDef
- сделать CreateWorkstationUIState(...) через WorkstationDatabase
- сделать HandleCraftAction(...)
- сделать HandleRepairAction(...)
- сделать HandleModifyAction(...)
- сделать HandleScrapAction(...)
- сделать HandleDiagnoseAction(...)
- сделать HandleAccessTerminalAction(...)
- сделать HandleOpenStorageAction(...)
- сделать общий dispatcher по StationActionType
- сделать вывод списка actions в OpenWorkstationUI(...)
- сделать вывод списка recipes в OpenWorkstationUI(...)
- сделать пустой выбор action
- сделать пустой выбор recipe
- убрать тестовый автокрафт
- не вызывать реальный крафт автоматически
- сохранить OpenWorkstationUI(...) как каркас станции
- подготовить ArmorWorkbench под Craft
- подготовить ArmorWorkbench под Repair

- подготовить ArmorWorkbench под Modify
- подготовить ArmorWorkbench под Scrap
- подготовить WeaponsWorkbench под Craft
- подготовить WeaponsWorkbench под Repair
- подготовить WeaponsWorkbench под Modify
- подготовить WeaponsWorkbench под Scrap
- подготовить TinkerWorkbench под Craft
- подготовить TinkerWorkbench под Scrap
- подготовить ChemStation под Craft
- подготовить TankMaintenanceBay под Repair
- подготовить TankMaintenanceBay под Modify
- подготовить TankMaintenanceBay под Diagnose
- подготовить Terminal под AccessTerminal
- подготовить Terminal под Diagnose
- подготовить Storage под OpenStorage
- сделать station-specific status messages
- сделать station-specific placeholder flow
- привязать recipes к station actions
- не смешивать TankMaintenanceBay с ArmorWorkbench
- не делать отдельную ехо-станцию
- оставить ArmorWorkbench для брони + экзоскелета
- расширить CraftingRecipe при необходимости под action/category
- сделать выбор рецепта по индексу
- сделать поиск реального recipe index
- проверить существование recipe перед запуском
- сделать реальную проверку requiredScrap
- сделать реальную проверку requiredCircuits
- сделать реальную проверку requiredCoreEnergy
- сделать списание Scrap Metal

- сделать списание Circuits
- сделать списание Core Energy
- добавить защиту от частичного списания ресурсов
- сделать возврат ошибки при невалидном recipe index
- сделать возврат ошибки при нехватке ресурсов
- сделать возврат сообщения об успешном крафте
- добавить station checks перед action execution
- проверять isActive
- проверять isDestroyed
- проверять isPowered
- учитывать requiresPower
- учитывать powerConsumption
- подключить WorldWorkstation к открытию UI
- искать WorkstationDef по objectID
- открывать нужный UI по stationType
- сделать player interaction со станцией в мире
- сделать открытие UI при взаимодействии с объектом станции
- хранить список WorldWorkstation
- спавнить WorldWorkstation
- хранить position станции
- хранить currentHealth станции
- хранить maxHealth станции
- хранить isPowered станции
- хранить isDestroyed станции
- хранить isActive станции
- добавить рендер station object
- привязать spriteTag к визуалу станции
- сделать визуально разные станции
- подключить assets/workstations/... к рендеру/спавну

- сделать build menu для CAMP/AIMP
- сделать категории строительства станций
- сделать выбор станции для постройки
- сделать проверку build cost
- сделать списание ресурсов на постройку
- сделать placement preview
- создать WorldWorkstation после постройки
- сохранять построенные станции
- загружать построенные станции
- сохранять состояние station health
- сохранять состояние station power
- сохранять состояние station activity
- сохранять состояние station type/objectID
- сделать Storage как stash/container UI
- сделать хранение содержимого Storage
- сделать Terminal как terminal UI
- сделать Terminal diagnostics flow
- сделать Terminal access flow
- сделать отдельный TankMaintenanceBay service flow
- сделать диагностику модулей БТ-7274
- сделать ремонт модулей БТ-7274
- сделать модификацию модулей БТ-7274
- сделать отображение статуса модулей БТ-7274
- добавить recipes для WeaponsWorkbench
- добавить recipes для TinkerWorkbench
- добавить recipes для ChemStation
- добавить recipes для TankMaintenanceBay
- разнести recipes по station type
- сделать station-aware UI output

- сделать station-aware action routing
- сделать station-aware gameplay loop
- связать station UI с инвентарем игрока
- связать station UI с GameState
- связать station UI с world interaction
- убрать временные заглушки после подключения логики
- заменить placeholder actions на реальную логику
- заменить placeholder status messages на реальные результаты
- довести flow до: build -> place -> interact -> action select -> recipe select -> execute -> result -> save

Ниже — полный список того, что осталось сделать, уже с пояснениями, но без воды.

- **расширить** WorkstationUIState
Сейчас это слишком маленькая структура. Она должна стать центральным контейнером состояния интерфейса станции.
- **добавить** supportedActions в WorkstationUIState
UI должен хранить список действий, доступных именно этой станции: Craft, Repair, Modify, Scrap, Diagnose, AccessTerminal, OpenStorage.
- **добавить** selectedActionIndex в WorkstationUIState
Нужен индекс выбранного действия, чтобы можно было переключаться между режимами станции, а не только между рецептами.
- **добавить** selectedRecipeIndex в WorkstationUIState
Нужен индекс выбранного рецепта внутри режима Craft.
- **добавить** statusMessage в WorkstationUIState
UI должен показывать результат последнего действия: успех, нехватка ресурсов, недоступность станции, заглушка и т.д.
- **подтягивать** uiTitle из WorkstationDatabase
Заголовок станции должен идти из данных станции, а не быть захардкожен в UI.
- **подтягивать** supportedActions из WorkstationDatabase
Доступные действия должны определяться WorkstationDef, чтобы логика станции шла из контента, а не из ручного switch.
- **убрать главный switch как основной источник UI-данных**
switch можно оставить только как fallback, но не как главный способ определения названия и функций станции.

- **оставить fallback на случай отсутствия WorkstationDef**
Если станция по objectID или типу не найдена, UI все равно должен открыться с базовым названием и пустым набором действий, а не падать.
- **сделать CreateWorkstationUIState(...) через WorkstationDatabase**
Эта функция должна собирать состояние UI из определения станции: title, actions, recipes, базовые флаги.
- **сделать HandleCraftAction(...)**
Заглушка или рабочая функция для режима крафта. Сначала просто маршрутизация и статус, потом подключение реального крафта.
- **сделать HandleRepairAction(...)**
Отдельный маршрут действия ремонта. Это нужно, чтобы ремонт был действием станции, а не отдельной станцией.
- **сделать HandleModifyAction(...)**
Отдельный маршрут для модификации экипировки, оружия или модулей танка.
- **сделать HandleScrapAction(...)**
Отдельный маршрут для разборки предметов на ресурсы.
- **сделать HandleDiagnoseAction(...)**
Нужен для терминала и TankMaintenanceBay, чтобы можно было показывать техническое состояние объектов.
- **сделать HandleAccessTerminalAction(...)**
Нужен как отдельная точка входа для terminal logic.
- **сделать HandleOpenStorageAction(...)**
Нужен как отдельная точка входа для stash/container UI.
- **сделать общий dispatcher по StationActionType**
Нужна функция, которая получает текущее действие и отправляет его в нужный обработчик. Это центральная развилка station logic.
- **сделать вывод списка actions в OpenWorkstationUI(...)**
Интерфейс станции должен сначала показывать, что вообще можно делать на этой станции.
- **сделать вывод списка recipes в OpenWorkstationUI(...)**
Если выбрано действие Craft, UI должен показывать список рецептов именно для этой станции.
- **сделать пустой выбор action**
Пока без полноценного UI, но уже нужен хотя бы базовый механизм выбора действия.

- **сделать пустой выбор recipe**
Точно так же нужен базовый механизм выбора рецепта без реального интерактивного интерфейса.
- **убрать тестовый автокрафт**
UI не должен автоматически крафтить первый рецепт при открытии. Это временный костыль, его надо убрать.
- **не вызывать реальный крафт автоматически**
Любое действие станции должно запускаться только через выбранное действие/рецепт, а не само по себе.
- **сохранить OpenWorkstationUI(...) как каркас станции**
Эта функция должна стать общей точкой входа для всех станций, а не отдельным тестовым print.
- **подготовить ArmorWorkbench под Craft**
На этой станции должен быть режим крафта брони и экзоскелета.
- **подготовить ArmorWorkbench под Repair**
Ремонт брони и экзоскелета должен происходить здесь же.
- **подготовить ArmorWorkbench под Modify**
Модификации брони/экзоскелета тоже должны быть частью этой станции.
- **подготовить ArmorWorkbench под Scrap**
Разбор защитного снаряжения — тоже действие этой станции.
- **подготовить WeaponsWorkbench под Craft**
Нужен отдельный режим крафта оружия.
- **подготовить WeaponsWorkbench под Repair**
Ремонт оружия должен идти через оружейный верстак.
- **подготовить WeaponsWorkbench под Modify**
Модификации оружия тоже должны быть здесь.
- **подготовить WeaponsWorkbench под Scrap**
Разбор оружия — тоже отдельное действие оружейной станции.
- **подготовить TinkerWorkbench под Craft**
Здесь должны собираться боеприпасы, гранаты, мины, технические утилиты.
- **подготовить TinkerWorkbench под Scrap**
У этой станции разборка будет полезна для возврата компонентов.
- **подготовить ChemStation под Craft**
Здесь должно быть создание медицины, химии, анти-эрозионных и похожих расходников.

- **подготовить TankMaintenanceBay под Repair**
Это сервисная станция танка/BT-7274, и ремонт — одна из ее основных функций.
- **подготовить TankMaintenanceBay под Modify**
Замена и улучшение модулей танка должны происходить именно здесь.
- **подготовить TankMaintenanceBay под Diagnose**
Нужна диагностика состояния частей танка: гусеницы, реактор, броня, оружие.
- **подготовить Terminal под AccessTerminal**
Терминал не должен быть просто “станцией без рецептов”; у него должен быть отдельный режим доступа.
- **подготовить Terminal под Diagnose**
Терминал может показывать состояние базы, питания, защитных систем, сетки и т.д.
- **подготовить Storage под OpenStorage**
Это не крафт-станция, а контейнерный интерфейс.
- **сделать station-specific status messages**
Сообщения должны зависеть от станции и действия: например, у terminal — один стиль сообщений, у tank bay — другой.
- **сделать station-specific placeholder flow**
Даже до подключения реальной логики разные станции должны вести себя как разные интерфейсы, а не одинаковые print-экраны.
- **привязать recipes к station actions**
Рецепты не должны показываться всегда. Они должны быть частью действия Craft.
- **не смешивать TankMaintenanceBay с ArmorWorkbench**
Танк/BT-7274 — отдельное направление логики, не часть станции брони.
- **не делать отдельную echo-станцию**
Экзоскелет должен обслуживаться на ArmorWorkbench, а не на отдельном станке.
- **оставить ArmorWorkbench для брони + экзоскелета**
Это финальная привязка роли этой станции.
- **расширить CraftingRecipe при необходимости под action/category**
Если позже понадобится, рецепт можно дополнить категорией, подтипом или action binding.
- **сделать выбор рецепта по индексу**
Для UI нужен нормальный выбор рецепта, а не автозапуск первого.
- **сделать поиск реального recipe index**
Если UI работает с фильтрованным списком рецептов станции, нужен способ сопоставить локальный индекс с индексом в общей базе рецептов.

- **проверить существование recipe перед запуском**
Перед крафтом должен быть защитный код от невалидного индекса.
- **сделать реальную проверку requiredScrap**
Сейчас проверка ресурсов должна опираться на поле рецепта, а не на условный hasItem(201).
- **сделать реальную проверку requiredCircuits**
Аналогично для электронных компонентов.
- **сделать реальную проверку requiredCoreEnergy**
Если рецепт требует энергоресурс, это тоже должно проверяться.
- **сделать списание Scrap Metal**
После успешной проверки scrap нужно реально вычитать из инвентаря.
- **сделать списание Circuits**
То же самое для circuits.
- **сделать списание Core Energy**
То же самое для core energy.
- **добавить защиту от частичного списания ресурсов**
Если одно списание прошло, а второе нет, нельзя оставлять систему в поломанном состоянии. Нужен безопасный порядок операций.
- **сделать возврат ошибки при невалидном recipe index**
Крафт не должен молча проваливаться.
- **сделать возврат ошибки при нехватке ресурсов**
Нужно различать причину отказа.
- **сделать возврат сообщения об успешном крафте**
После выполнения игрок должен получить понятный результат.
- **добавить station checks перед action execution**
Перед выполнением действия нужно проверить состояние станции.
- **проверять isActive**
Неактивная станция не должна выполнять действия.
- **проверять isDestroyed**
Разрушенная станция не должна открываться как рабочая.
- **проверять isPowered**
Для станций, зависящих от питания, это обязательная проверка.
- **учитывать requiresPower**
Не все станции обязаны требовать питание, но если у станции это указано, логика должна это уважать.

- **учитывать powerConsumption**
Позже это потребуется для базы/энергосистемы и ограничения работы построек.
- **подключить WorldWorkstation к открытию UI**
Интерфейс станции должен открываться не от “абстрактного типа”, а от реального объекта в мире.
- **искать WorkstationDef по objectID**
Именно так world object должен находить свое контентное определение.
- **открывать нужный UI по stationType**
После нахождения def должен открываться правильный station flow.
- **сделать player interaction со станцией в мире**
Игрок должен физически взаимодействовать с объектом станции, а не вызывать UI вручную из теста.
- **сделать открытие UI при взаимодействии с объектом станции**
Нужен полный путь: подошел → нажал взаимодействие → открылся нужный UI.
- **хранить список WorldWorkstation**
Все размещенные станции должны существовать как объекты мира.
- **спавнить WorldWorkstation**
После постройки или загрузки игры станции должны появляться в мире.
- **хранить position станции**
Нужна координата размещения в мире.
- **хранить currentHealth станции**
Станции должны иметь повреждаемое состояние.
- **хранить maxHealth станции**
Нужен верхний предел прочности.
- **хранить isPowered станции**
Станция должна знать, подано ли на нее питание.
- **хранить isDestroyed станции**
Это влияет на доступность интерфейса и дальнейший ремонт.
- **хранить isActive станции**
Это отдельный флаг работоспособности.
- **добавить рендер station object**
Станция должна быть не только логикой, но и видимым world-объектом.
- **привязать spriteTag к визуалу станции**
Поле уже есть в данных, но нужно реально использовать его при отображении.

- **сделать визуально разные станции**
Armor, Weapons, Tinker, Chem, Tank, Terminal, Storage должны быть отличимы визуально.
- **подключить assets/workstations/... к рендеру/спавну**
Нужно реально использовать подготовленную структуру ассетов.
- **сделать build menu для CAMP/AIMP**
Так как станции строятся игроком, нужен интерфейс строительства.
- **сделать категории строительства станций**
Чтобы игрок выбирал их как отдельные объекты CAMP/AIMP.
- **сделать выбор станции для постройки**
Нужен список buildable station objects.
- **сделать проверку build cost**
Перед установкой станции нужно проверить ресурсы.
- **сделать списание ресурсов на постройку**
После успешной установки стоимость должна списываться.
- **сделать placement preview**
Игрок должен видеть, куда ставится станция.
- **создать WorldWorkstation после постройки**
Постройка должна порождать реальный объект станции в мире.
- **сохранять построенные станции**
Станции игрока должны переживать выход из игры.
- **загружать построенные станции**
При старте игры нужно восстанавливать их.
- **сохранять состояние station health**
Станции должны возвращаться с тем же уровнем повреждений.
- **сохранять состояние station power**
Состояние питания тоже должно сохраняться.
- **сохранять состояние station activity**
Активность/деактивация станции должна переживать сохранение.
- **сохранять состояние station type/objectID**
Без этого невозможно восстановить правильный station def.
- **сделать Storage как stash/container UI**
Хранилище должно открывать инвентарь контейнера, а не крафт-меню.
- **сделать хранение содержимого Storage**
Контейнер должен иметь собственные предметы.

- **сделать Terminal как terminal UI**
Терминал должен открывать отдельный экран с терминальной логикой.
- **сделать Terminal diagnostics flow**
Через терминал можно выводить данные о системах базы.
- **сделать Terminal access flow**
Через терминал можно открывать доступ к функциям или сюжетным блокам.
- **сделать отдельный TankMaintenanceBay service flow**
Это отдельная станция с собственной сервисной логикой, не крафт-верстак брони.
- **сделать диагностику модулей БТ-7274**
Нужно уметь выводить статус частей танка по их HP/состоянию.
- **сделать ремонт модулей БТ-7274**
Должна быть логика восстановления поврежденных частей.
- **сделать модификацию модулей БТ-7274**
Нужно отдельное направление улучшения/замены модулей танка.
- **сделать отображение статуса модулей БТ-7274**
Игрок должен видеть состояние tracks, reactor, cannon, armor и т.д.
- **добавить recipes для WeaponsWorkbench**
Сейчас рецепты в основном на armor; оружейная станция должна получить свои.
- **добавить recipes для TinkerWorkbench**
Нужны рецепты техпредметов, гранат, боеприпасов, утилит.
- **добавить recipes для ChemStation**
Нужны рецепты медикаментов, химии и защитных расходников.
- **добавить recipes для TankMaintenanceBay**
Если через bay будут собираться или обслуживаться модули/комплекты, это тоже надо оформить рецептами или service entries.
- **разнести recipes по station type**
Каждая станция должна видеть только свои рецепты.
- **сделать station-aware UI output**
Интерфейс должен зависеть от типа станции и ее доступных действий.
- **сделать station-aware action routing**
Разные станции должны вести в разные обработчики и разные ветки логики.
- **сделать station-aware gameplay loop**
Весь цикл работы должен зависеть от того, с какой станцией взаимодействует игрок.

- **связать station UI с инвентарем игрока**
Без этого нельзя делать реальный крафт, ремонт, разбор, перенос предметов.
- **связать station UI с GameState**
Это нужно для общего состояния игры, танка, карты, базы, прогрессии и т.д.
- **связать station UI с world interaction**
UI должен запускаться не тестовой функцией, а через мир.
- **убрать временные заглушки после подключения логики**
Когда маршрутизация и интерфейс будут готовы, placeholder-функции нужно заменить реальной механикой.
- **заменить placeholder actions на реальную логику**
Каждый action handler должен выполнять настоящее действие, а не только выводить статус.
- **заменить placeholder status messages на реальные результаты**
Вместо “logic placeholder” должны появиться настоящие сообщения по факту выполнения действия.
- **довести flow до: build -> place -> interact -> action select -> recipe select -> execute -> result -> save**
Это итоговый законченный цикл station system, к которому все должно прийти.

- Сейчас у тебя уже есть база станций, типы станций, набор действий, `WorkstationUIState`, рецепты и первичный UI-хук. Но до нормальной рабочей системы не хватает трех больших слоев: **правильного UI-каркаса**, **реальной логики действий станции**, и **привязки станции к миру/строительству/сохранению**.
- Первое, что нужно доделать — это сам **каркас интерфейса станции**. `WorkstationUI` сейчас должен перестать быть просто функцией, которая печатает название и список рецептов. Он должен стать нормальной оболочкой станции. Для этого `WorkstationUIState` надо расширить: хранить не только `stationType`, `title` и список рецептов, но и список доступных действий (`supportedActions`), индекс выбранного действия, индекс выбранного рецепта, а также `statusMessage`. После этого `CreateWorkstationUIState(...)` должен собирать состояние не из ручного `switch`, а из `WorkstationDatabase`: брать `uiTitle`, `supportedActions`, тип станции и рецепты для этой станции. `switch` можно оставить только как запасной fallback, если определение станции не найдено. Когда это будет сделано, `OpenWorkstationUI(...)` должен уже показывать не просто рецепты, а сначала саму станцию, потом доступные для нее

действия, и уже потом — если текущее действие `Craft` — выводить список рецептов.

- Следующий слой — это **маршрутизация действий станции**. У тебя уже есть `StationActionType`, и это надо наконец превратить в реальную систему. Нужен общий dispatcher: если выбрано `Craft`, вызывается один обработчик; если `Repair` — другой; если `Modify` — третий; если `Diagnose` — четвертый, и так далее. На первом этапе эти обработчики могут быть пустыми заглушками, но сама структура должна быть уже правильной. То есть должны появиться отдельные функции под `HandleCraftAction(...)`, `HandleRepairAction(...)`, `HandleModifyAction(...)`, `HandleScrapAction(...)`, `HandleDiagnoseAction(...)`, `HandleAccessTerminalAction(...)`, `HandleOpenStorageAction(...)`. Даже если они пока только ставят `statusMessage = "Logic placeholder"`, это уже создаст правильную архитектуру: одна станция — много действий, и каждое действие живет своей веткой, а не мешается в одном куске `cout`.
- После этого нужно включить **реальную логику крафта**, но аккуратно. Сейчас `craftItem(...)` еще слишком примитивен и проверяет только наличие одного предмета по ID. Его нужно довести до нормального состояния: проверять `requiredScrap`, `requiredCircuits`, `requiredCoreEnergy`, валидность индекса рецепта, и потом списывать ресурсы перед выдачей результата. Причем делать это надо безопасно: если ресурсов не хватает, сразу возвращать ошибку; если индекс рецепта битый — тоже ошибка; если одно списание прошло, а второе нет — не оставлять систему в поломанном частичном состоянии. Только после успешной проверки и успешного списания должен создаваться предмет через `addItem(...)`. Параллельно UI должен уметь показывать игроку, почему крафт не прошел: не хватает scrap, не хватает circuits, не хватает энергии, станция неактивна и т.п.
- Дальше важный шаг — **развести действия по станциям**, а не делать вид, что все станции одинаковые. `ArmorWorkbench` должен обслуживать броню и экзоскелет: крафт, ремонт, модификацию и разбор. `WeaponsWorkbench` — оружие, тоже с крафтом, ремонтом, модификацией и разбором. `TinkerWorkbench` — боеприпасы, гранаты, утилиты и технический крафт, плюс, при желании, разбор. `ChemStation` — медикаменты, химия и защитные расходники. `TankMaintenanceBay` — отдельная тяжелая сервисная станция для BT-7274/танка: диагностика, ремонт модулей, модификация модулей, сервис корпуса, энергосистемы и т.д. `Terminal` должен открывать терминальный интерфейс,

а `Storage` — stash/container UI. Это значит, что даже до реальной логики каждая станция должна уже иметь **свой набор действий** и свой **сценарий поведения**, а не просто разное название.

- Параллельно нужно довести до конца **контентную сторону рецептов**. Сейчас рецепты в основном сидят на `ArmorWorkbench`, а остальные станции остаются пустыми или почти пустыми. Это надо разнести: добавить рецепты для `WeaponsWorkbench`, `TinkerWorkbench`, `ChemStation`, при необходимости для `TankMaintenanceBay`, и четко привязать каждый рецепт к `stationType`. Тогда UI будет вести себя правильно сам по себе: открыл station → получил только те рецепты, которые должны быть у этой станции.
- После этого надо подключить **состояние самой станции в мире**. У тебя уже есть `WorldWorkstation` с полями `isActive`, `isPowered`, `isDestroyed`, `currentHealth`, `maxHealth`. Пока это почти не используется. Нужно, чтобы перед выполнением любого действия станция проверялась: если она разрушена — интерфейс либо не открывается, либо открывается с сообщением, что объект уничтожен; если станция требует питание и оно не подано — часть действий недоступна; если `isActive == false`, то она тоже не должна работать. Это особенно важно для `TankMaintenanceBay`, терминалов и других `powered`-объектов. То есть UI и station logic должны работать не “в вакууме”, а от состояния реального объекта в мире.
- Следующий большой блок — **world interaction**. Сейчас станцию можно открыть только как абстрактный тип, а должно быть так: в мире стоит объект станции, у него есть `objectID`, по нему находится `WorkstationDef`, из него берется `stationType`, и дальше открывается правильный UI. Это значит, что надо определить место в проекте, где у тебя идет обработка взаимодействия игрока с миром, и именно туда вшить station flow. Игрок подходит к объекту, нажимает кнопку, система понимает, что это workstation, находит ее def и открывает нужный экран.
- После этого уже нужно идти в сторону **реального размещения станций**, потому что по дизайну они у тебя CAMP/AIMP-постройки. Значит, надо хранить список `WorldWorkstation`, уметь создавать их при постройке, размещать по координатам, учитывать габариты, радиус взаимодействия, питание, здоровье, разрушение. Иными словами, станции должны перестать быть просто “данными в базе” и стать полноценными world-объектами. Для этого понадобится build menu, категории строительства, выбор станции для постройки,

предпросмотр установки, проверка стоимости, списание ресурсов на постройку и создание `WorldWorkstation` после подтверждения.

- Как только станции появятся в мире как реальные объекты, надо будет подключить **их визуальную часть**. У тебя уже есть `spriteTag` и структура ассетов в `assets/workstations/...`, но это пока контентный слой без полноценного использования. Нужно, чтобы при спавне станции по `spriteTag` подтягивался нужный визуал. И станции должны действительно выглядеть по-разному: `ArmorWorkbench` не должен выглядеть как `TankMaintenanceBay`, `terminal` — как `storage`, и так далее. Это важно, потому что пользовательский замысел как раз был в стиле *Fallout 76*: разные станции, разные *silhouettes*, разные *roles*.
- Далее идет **специальная логика для нестандартных станций**. `Storage` не должен открывать крафт-рецепты — это должен быть `stash/container UI` с собственным содержимым. `Terminal` не должен открывать список `armor recipes` — у него должен быть `terminal screen`, через который идет доступ, диагностика, возможно управление базой или сюжетные функции. `TankMaintenanceBay` не должен притворяться обычным верстаком — ему нужен отдельный `service flow` с показом состояния модулей *BT-7274*, ремонтом, диагностикой и модификациями. То есть после общего каркаса станции тебе надо будет сделать отдельные ветки логики для `terminal/storage/tank bay`, потому что именно они сильнее всего отличаются от обычного `crafting bench`.
- После этого останется **`persistence`**. Раз станции строятся и живут в мире, их недостаточно просто иметь в памяти во время одной сессии. Нужно сохранять список `WorldWorkstation`: их тип, `objectID`, позицию, здоровье, питание, активность, состояние разрушения. Потом при загрузке эти данные должны восстанавливаться и снова превращаться в реальные объекты на карте. Иначе *CAMP/AIMP*-станции не будут ощущаться как часть базы игрока.
- И уже финальный этап — это связать все в **единый station gameplay loop**. Итоговая цепочка должна выглядеть так: игрок открывает `build menu`, выбирает станцию, ставит ее в *CAMP/AIMP*, тратит ресурсы, станция появляется в мире, игрок подходит к ней, взаимодействует, открывается именно ее `UI`, `UI` показывает доступные действия, игрок выбирает действие, при необходимости выбирает рецепт, система проверяет состояние станции и ресурсы игрока, выполняет действие, показывает результат, и после этого состояние станции и мира может быть сохранено. Когда у тебя заработает именно этот цикл, тогда `station system` можно будет

считать реально внедренной, а не просто лежащей набором типов и файлов в проекте.

- Если хочешь, следующим сообщением я уже не буду объяснять архитектуру, а дам **первый правильный пустой патч поверх v.109** — только `WorkstationUI.hpp` и `WorkstationUI.cpp`, чтобы у тебя появился каркас действий станции без поломки остального кода.

Осталось довести систему станций от текущего состояния, где уже есть типы станций, база определений, `WorldWorkstation`, рецепты, `WorkstationUIState` и тестовый UI, до полноценного рабочего цикла, который мы и обговорили. То есть не ломать текущую структуру, не пытаться сразу подключить всё подряд, а поэтапно превратить это в нормальную *Fallout 76*–style station system.

Сначала нужно оставить и расширить `WorkstationUIState`, чтобы он стал не просто контейнером под заголовок и рецепты, а реальным состоянием интерфейса станции. В нём должны быть доступные действия станции, текущий выбранный action, текущий выбранный recipe и строка статуса. После этого `WorkstationUI` должен строиться уже не через ручной switch как основной источник данных, а через `WorkstationDatabase`: брать `uiTitle`, брать `supportedActions`, брать тип станции, а switch оставить только как запасной fallback. Это нужно для того, чтобы интерфейс шёл от реальных station definitions, а не от временно захардкоженной логики.

Дальше нужно превратить `StationActionType` из просто enum'a в реально используемую систему действий. Мы договорились, что сейчас лучше сделать пустую, но правильную архитектуру: создать отдельные обработчики под `Craft`, `Repair`, `Modify`, `Scrap`, `Diagnose`, `AccessTerminal`, `OpenStorage`, и сделать общий dispatcher, который будет направлять выбранное действие в нужную ветку. На первом этапе эти функции могут быть заглушками с `statusMessage`, но сама структура должна уже быть нормальной. Это и есть тот “пустой каркас”, который мы решили делать сначала, а уже потом наполнять логикой.

После этого `OpenWorkstationUI(...)` должен перестать быть просто функцией вывода списка рецептов. Он должен открывать станцию как оболочку: показывать заголовок станции, показывать список действий, показывать рецепты, если выбрано `Craft`, и выводить статус выполнения или заглушки. При этом мы договорились убрать тестовый автокрафт и не запускать никакое действие автоматически. Сначала нужен именно каркас станции, а не костыль с мгновенным вызовом первого рецепта.

Дальше уже надо подключать реальный крафт, но не ломая остальную систему. `craftItem(...)` должен перестать быть заглушкой с `hasItem(201)` и начать работать по данным рецепта: проверять `requiredScrap`, `requiredCircuits`, `requiredCoreEnergy`, проверять валидность рецепта, списывать ресурсы и только после этого выдавать результат. При этом UI должен уметь показывать понятный статус: что именно не хватило, что именно было создано или почему действие не выполнено.

Параллельно нужно довести различие между самими станциями до рабочего уровня. ArmorWorkbench должен остаться станцией для брони и экзоскелета — без отдельной echo-станции. WeaponsWorkbench должен стать местом для оружия. TinkerWorkbench — для патронов, гранат, мин и утилитарного технического крафта. ChemStation — для медицины и химии. TankMaintenanceBay должен остаться отдельно и не смешиваться с ArmorWorkbench: у него свой набор действий, отдельная логика ремонта, модификации и диагностики BT-7274/танка. Terminal и Storage тоже не должны быть фейковыми крафт-станциями: у первого должен быть terminal flow, у второго — stash/container flow.

После этого нужно разнести и наполнить контент по станциям. Сейчас рецепты в основном сидят на ArmorWorkbench, а остальные станции почти пустые. Их нужно развести по stationType: добавить рецепты для WeaponsWorkbench, TinkerWorkbench, ChemStation, при необходимости отдельные recipe/service entries для TankMaintenanceBay. Тогда станции начнут отличаться не только названием, но и реальным содержимым.

Отдельно мы обговорили, что дальше надо подключить WorldWorkstation и состояние реальной станции в мире. У каждой станции должно учитываться, активна ли она, разрушена ли она, есть ли у неё питание и какое у неё текущее здоровье. То есть перед выполнением действия должен быть station check: если станция уничтожена — не использовать; если питание обязательно, а его нет — действие не выполнять; если станция неактивна — тоже не выполнять. Иначе интерфейс станции будет оторван от реального объекта в мире.

После этого потребуется полноценная world interaction логика. Игрок должен взаимодействовать не с абстрактным WorkstationType, а с реальным объектом станции в мире. Нужно находить WorkstationDef по objectID world-объекта, определять тип станции и открывать именно нужный station UI. Это и есть правильный путь “подошел к станции → открыл её интерфейс”, который мы и хотели.

Дальше уже подключается этап реального CAMP/AIMP-плейсмента. Поскольку станции у тебя по дизайну строятся игроком, надо сделать build flow: build menu, выбор станции, предпросмотр размещения, проверка стоимости, списание ресурсов, создание WorldWorkstation, и дальше — возможность потом взаимодействовать с этой построенной станцией как с обычным объектом мира. То есть станция должна не просто существовать как определение в коде, а реально ставиться игроком в мире.

Потом нужен визуальный слой: у тебя уже есть spriteTag и папка assets/workstations/..., значит надо связать spriteTag с рендером/спавном. Каждая станция должна выглядеть по-разному, как мы и обсуждали: ArmorWorkbench, WeaponsWorkbench, ChemStation, TankMaintenanceBay, Terminal, Storage — всё это должны быть разные визуальные объекты, а не одна и та же заглушка.

Дальше пойдут специальные ветки логики. Storage нужно превратить в контейнер/тайник с хранением содержимого. Terminal — в отдельный интерфейс доступа и диагностики. TankMaintenanceBay — в полноценную сервисную станцию с показом состояния модулей BT-7274, ремонтом, модификациями и диагностикой. Это уже не общий крафт-каркас, а отдельные ветки station gameplay, которые должны вырасти поверх общей системы действий.

И наконец, всё это надо сохранить в persistence. Раз станции строятся и живут в мире, нужно сохранять их тип, позицию, здоровье, питание, активность и другие важные флаги, а потом восстанавливать при загрузке. Иначе CAMP/AIMP-станции не будут ощущаться как часть базы игрока.

То есть в сумме мы договорились о такой последовательности: сначала делаем пустой, но правильный WorkstationUI-каркас с supportedActions, выбором action, выбором recipe и пустыми обработчиками; потом подключаем реальный craftItem(...); потом подключаем отличия станций и их рецепты; потом связываем всё с WorldWorkstation и interaction; потом делаем build/place flow; потом визуал, terminal/storage/tank service logic; и в конце — сохранение и полный station gameplay loop. Если хочешь, следующим сообщением я могу уже превратить именно этот план в **конкретный порядок файлов, которые надо менять по шагам**, без общих слов.